

Generalized pack declaration and usage

Document #: P1858R0
Date: 2019-10-06
Project: Programming Language C++
EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

§ 1 Introduction and Motivation

C++11 introduced variadic templates, one of the truly transformational language features introduced that standard. Despite pretty tight restrictions on where packs could be declared and how they could be used, this feature has proven incredibly successful. Three standards later, there hasn't even been much change. C++17 added a couple new ways to use packs (fold expressions [\[N4191\]](#) and using-declarations [\[P0195R2\]](#)), and C++20 will add a new way to introduce them (in lambda capture [\[P0780R2\]](#)). A proposal to iterate over them (expansion statements [\[P1306R1\]](#)) didn't quite make it. That's it.

There have been many papers in the interlude about trying to enhance pack functionality: a language typelist [\[N3728\]](#), fixed size and homogeneous packs [\[N4072\]](#) (and later [\[P1219R1\]](#)), indexing and slicing into packs [\[N4235\]](#) and [\[P0535R0\]](#), being able to declare packs in more places [\[P0341R0\]](#) and other places [\[P1061R0\]](#).

In short, there's been work in this space, although not all of these papers have been discussed by Evolution. Although, many of these have been received favorably and then never followed up on.

Yet the features that keep getting hinted at and requested again and again are still missing from our feature set:

1. the ability to declare a variable pack at class, namespace, or local scope
2. the ability to index into a pack
3. the ability to unpack a tuple, or tuple-like type, inline

All efficiently, from a compile time perspective. Instead, for (1) we have to use `std::tuple`, for (2) we have to use `std::get`, `std::tuple_element`, or, if we're only dealing with types, something like `mp_at_c` [\[Boost.Mp11\]](#), for (3) we have to use `std::apply` (`()`), which necessarily introduces a new scope. `std::apply` (`()`) is actually worse than that, since it doesn't play well with callables that aren't objects and even worse if you want to use additional arguments on top of that:

```
std::tuple args(1, 2, 3);

// I want to call f with args... and then 4
std::apply([&](auto... vs){ return f(vs..., 4); }, args);
```

Matt Calabrese is working on a library facility to help address the shortcomings here [\[Calabrese.Argot\]](#).

This paper attempts to provide a solution to these problems, building on the work of prior paper authors. The goal of this paper is to provide a better implementation for a library `tuple`, one that ends up being much easier to implement, more compiler friendly, and more ergonomic. The paper will piecewise introduce the necessary language features, increasing in complexity as it goes.

§ 2 The Tuple Example

This paper proposes the ability to declare a variable pack wherever we can declare a variable today:

```
namespace xstd {
    template <typename... Ts>
    struct tuple {
        Ts... elems;
    };
}
```

That gives us all the members that we need, using a syntax that arguably has obvious meaning to any reader familiar with C++ packs. All the usual rules follow directly from there. That class template is an aggregate, so we can use aggregate initialization:

```
xstd::tuple<int, int, int> x{1, 2, 3};
```

Or, in C++20:

```
xstd::tuple y{1, 2, 3};
```

§ 2.1 Empty variable pack

One question right off the back: what does `xstd::tuple<> t;` mean? The same way that an empty function parameter pack means a function taking no arguments, an empty member variable pack means no member variables.

`xstd::tuple<>` is an empty type.

§ 2.2 Constructors

But `tuple` has constructors. `tuple` has *lots* of constructors. We’re not going to go through all of them in this paper, just the interesting ones. But let’s at least start with the easy ones:

```
namespace xstd {
    template <typename... Ts>
    class tuple {
    public:
        constexpr tuple() requires (std::default_constructible<Ts> && ...)
            : elems()...
        { }

        constexpr tuple(Ts const&... args)
            requires (std::copy_constructible<Ts> && ...)
            : elems(args)...
        { }

    private:
        Ts... elems;
    };
}
```

Note the new pack expansion in the *mem-initializer*. This is a new ability this paper is proposing. It wouldn’t have made sense to have if you could not declare a member variable pack.

Let’s pick a more complex constructor. A `std::tuple<Ts...>` can be constructed from a `std::pair<T, U>` if `sizeof...(Ts) == 2` and the two corresponding types are convertible. How would we implement that? To do that check, we need to get the corresponding types. How do we get the first and second types from `Ts`?

§ 2.3 Pack Indexing

This paper proposes a “simple selection” facility similar to the one initially introduced in [\[N4235\]](#) (and favorably received in Urbana 2014): `T...[I]` is the `I`th element of the pack `T`, which is a type or value or template based on what kind of pack `T` is. [Later sections](#) of this paper will discuss why this paper diverges from that original proposal in choice of syntax.

Such indexing allows for implementing the pair converting constructor:

```
namespace xstd {
    template <typename... Ts>
    class tuple {
    public:
        template <std::convertible_to<Ts...[0]> T,
                  std::convertible_to<Ts...[1]> U>
            requires sizeof...(Ts) == 2
        constexpr tuple(std::pair<T, U> const& p)
            : elems...[0](p.first)
            , elems...[1](p.second)
        { }
    private:
        Ts... elems;
    };
}
```

A properly constrained converting constructor from a pack:

```
namespace xstd {
    template <typename... Ts>
    class tuple {
    public:
        template <std::constructible<Ts>... Us> // everything is convertible
            requires (sizeof...(Us) > 1 || // exclude the copy ctor match
                !std::derived_from<std::remove_cvref_t<Us...[0]>, tuple>)
            constexpr tuple(Us&&... us)
                : elems(std::forward<Us>(us))...
            { }
    private:
        Ts... elems;
    };
}
```

As well as implementing tuple_element:

```
namespace xstd {
    template <typename... > class tuple;

    template <size_t I, typename>
    struct tuple_element;
    template <size_t I, typename... Ts>
    struct tuple_element<I, tuple<Ts...>> requires (I < sizeof...(Ts))
    {
        using type = Ts...[I];
    };
}
```

This is nicer than status quo, but I think we can do better in this regard with a little bit more help.

§ 2.4 Pack aliases and generalized indexing

The previous section defines the syntax `T ...[I]` to be indexing into a pack, `T`. Let's immediately generalize this. Let's also say that a *type* can be pack-indexed into if the type provides an alias named `...`.

That is:

```
namespace xstd {
    template <typename... Ts>
    class tuple {
    public:
        using ... = Ts; // declares that tuple<Ts...> can be indexed just like Ts...
                        // note that the Ts on the right-hand-side is not expanded
    };

    template <size_t I, typename Tuple>
    struct tuple_element {
        using type = Tuple.[I]; // indexes via the pack Tuple:...
    };
}
```

The above is not a typo: while we index into a *pack* by way of `x ...[I]`, we index into a specific type or object via `x .[I]`. A proper discussion of the motivation for the differing syntax will follow.

This isn't quite right though, since we want to constrain `tuple_element` here. We have the operator `sizeof...`, which takes a pack. We just need a way of passing the pack itself. To do that, this paper proposes the syntax `T .[:]`. You can think of this as the “add a layer of packness” operator:

```
namespace xstd {
    template <typename... Ts>
    class tuple {
    public:
        using ... = Ts;
    };

    template <size_t I, typename Tuple>
    struct tuple_element;
```

```

template <size_t I, typename Tuple>
    requires (I < sizeof...(Tuple.[:]))
    struct tuple_element<I, Tuple>
    {
        using type = Tuple.[I];
    };
}

```

In the wild, we could use `Tuple.[I]` directly. It is SFINAE-friendly (simply discard the overload if the type either does not provide a pack template or the *constant-expression* `I` is out of bounds for the size of the pack).

§ 2.5 Named pack aliases

Pack aliases can be named as well, it's just that the unnamed pack alias gets special treatment as far as the language is concerned:

```

namespace xstd {
    template <typename... Ts>
    class tuple {
    public:
        using ... = Ts;
        using ...refs = Ts&;
    };

    using Record = tuple<int, double, std::string>;
    static_assert(std::is_same_v<Record.[0], int>);
    static_assert(std::is_same_v<Record::refs...[1], double&>);
}

```

Note the differing access syntax: `Record` is a type that we're treating as a pack, whereas `Record::refs` is a pack.

§ 2.6 Unpacking

Let's go back to our initial sketch, where `xstd::tuple` was an aggregate:

```

namespace xstd {
    template <typename... Ts>
    struct tuple {
        Ts... elems;
    };
}

```

Since we know `elems` is a pack, we can directly access and unpack it into a function:

```

int sum(int x, int y, int z) { return x + y + z; }

xstd::tuple<int, int, int> point{1, 2, 3};
int s = sum(point.elems...); // ok, 6

```

This can work since the compiler knows that `point.elems` is a pack, and unpacking that is reasonable following the rules of the language.

However, we quickly run into a problem as soon as we add more templates into the mix:

```

template <typename Tuple>
int tuple_sum(Tuple t) {
    return sum(t.elems...); // ??
}

```

This isn't going to work. From [\[Smith.Pack\]](#):

```

template<typename T> void call_f(T t) {
    f(t.x ...)
}

```

Right now, this is ill-formed (no diagnostic required) because “`t.x`” does not contain an unexpanded parameter pack. But if we allow class members to be pack expansions, this code could be valid – we'd lose any syntactic mechanism to determine whether an expression contains an unexpanded pack. This is fatal to at least one implementation strategy for variadic templates. It also

admits the possibility of pack expansions occurring outside templates, which current implementations are not well-suited to handle.

As well as introducing ambiguities:

```
template <typename T, typename... U>
void call_f(T t, U... us)
{
    // Is this intended to add 't' to each element in 'us'
    // or is intended to pairwise sum the tuple 't' and
    // the pack 'us'?
    f((t + us)...);
}
```

We can't have *no* syntactic mechanism (and note that this paper very much is introducing the possibility of pack expansions occurring outside templates). In order to make the dependent tuple_sum case work, we need one. One such could be a context-sensitive keyword like pack:

```
template <typename Tuple>
int tuple_sum(Tuple t) {
    return sum(t.pack elems...);
}
```

However, having a context-sensitive keyword isn't going to cut it... because of the possibility of writing code like this:

```
template<typename ...Ts> struct X { using ...types = Ts; };
template<typename MyX> void f() {
    using Fn = void(typename MyX::pack types ...);
}
```

Today that's declaring a function type that takes one argument of type `typename MyX` `::pack` named types and then varargs with the comma elided. If we make the comma mandatory (as [\[P1219R1\]](#) proposes to do), then that would open up our ability to use a context-sensitive pack here.

Otherwise, we would need a new keyword to make this happen, and pack seems entirely too pretty to make this work. As a placeholder, this paper suggests using *preceding* ellipses (which still need to be separated by a space). That is:

```
template <typename Tuple>
int tuple_sum(Tuple t) {
    return sum(t. ...elems...);
}
```

Class access wouldn't need a leading dot (e.g. `Tuple ::...elems`), but either way that's a lot of dots. Don't worry too much about the above syntax, it's not intended to be the commonly used approach - simply something that would be necessary to have. And such a marker would necessarily be an incomplete solution anyway. With the earlier examples of implementing tuple having constructors, `elems` was private. How would we access it?

§ 2.7 Generalized unpacking and `operator ...`

In the same way that we let `xstd::tuple` `<Ts ...>` be directly indexed into, we can also let it be directly unpacked. We do that with the help of `operator ...()`:

```
namespace xstd {
    template <typename... Ts>
    class tuple {
    public:
        Ts& operator ...() & { return elems; }
        Ts const& operator ...() const& { return elems; }
        Ts&& operator ...() && { return std::move(*this).elems; }
        Ts const&& operator ...() const&& { return std::move(*this).elems; }
    private:
        Ts... elems;
    };
}
```

We do not need to disambiguate `elems` here because we know `elems` is a pack, it's declared as such. Were we to retrieve `elems` from a dependent base class though, we would need some form of disambiguation as described above (i.e. `this ->...elems`).

Note that this form of the declaration uses an unexpanded pack on the left (the `const` `&`, not really anyway) and in the body. Later sections in the paper will show other forms.

Also, from here on out, this paper will only use the `const` `&` overloads of relevant functions for general sanity (see also [P0847R2]).

The above declarations allow for:

```
template <typename Tuple>
constexpr auto tuple_sum(Tuple const& tuple) {
    return (tuple[:] + ...);
}

static_assert(tuple_sum(xstd::tuple(1, 2, 3)) == 6);
```

`tuple` `[:]` works by adding a layer of packness on top of `tuple` - it does this by creating a pack by way of `operator` `...()` and using that pack as an unexpanded pack expression. That unexpanded pack is then expanded with the *fold-expression* as if it were a normal pack.

Or more generally:

```
template <typename F, typename Tuple>
constexpr decltype(auto) apply(F&& f, Tuple&& tuple) {
    return std::invoke(std::forward<F>(f), std::forward<Tuple>(tuple).[:]);
}
```

As well as writing our `get` function template:

```
namespace xstd {
    template <typename... Ts>
    class tuple {
    public:
        using ... = Ts;
        Ts const& operator ...() const { return elems; }
    private:
        Ts... elems;
    };

    template <size_t I, typename... Ts>
    constexpr auto const& get(tuple<Ts...> const& v) noexcept {
        return v.[I];
    }
}
```

Although since the syntax directly allows for `v` `.[` `0` `]`, why would anyone write `xstd` `::get` `<` `0` `>(v)`? And if you could write `f` `(t` `.[:]...`), why would anyone call `std` `::apply` `()`?

§ 2.8 Syntax-free unpacking?

The above allows us to write:

```
xstd::tuple x{1, 2, 3};
foo(x.[:]); // calls foo(1, 2, 3)
```

But do we explicitly need to add a layer of packness to `x` when we already know that `x` is a `tuple` and nothing is dependent? Could we simply allow:

```
foo(x...); // implicitly add packness to x and unpack it
```

I'm not sure it's worth it to pursue.

§ 2.9 Disambiguating packs of tuples

The previous section showed how to write `apply` taking a single function and a single tuple. What if we generalized it to taking multiple tuples? How do we handle a pack of tuples?

It's at this point that it's worth taking a step back and talking about disambiguation and why this paper makes the syntax choices that it makes. We need to be able to differentiate between packs and tuples. The two concepts are very similar, and this paper seeks to make them much more similar, but we still need to differentiate between them. It's the pack of tuples case that really brings the ambiguity to light.

The rules this paper proposes, which have all been introduced at this point, are:

- e `.[:]` takes a [pack-like type](#) (or object of such) and adds a layer of packness to it, by way of either `operator` `...()` or `using` `....`. It never is applied to an existing pack, and it is never used to disambiguate dependent member access.
- e `.[I]` never removes a layer of packness. It is picking the Ith element of a pack-like type.
- e `...[I]` always removes a layer of packness. It is picking the Ith element of a pack.
- e `...f` disambiguates dependent member access and identifies f as a pack. The space between the `.` and `...` is required.

That is, pack `...[I]` and tuple `.[I]` are valid, tuple `...[I]` is an error, and pack `.[I]` would be applying `.[I]` to each element of the pack (and is itself still an unexpanded pack expression). Rule of thumb: you need `...`s if and only if you have a pack.

e `.[I]` is an equivalent shorthand for e `.[:]....`.

This leads to clear meanings of each of the following. If we have a function template taking an argument e which has a member f, where the kind of e is specified by the columns of this table and the kind of f is specified by the rows:

	e is a Pack	e is a Pack-like type	e is not expanded
f is a Pack	foo (ef ...);	foo (e . [:]. ...f ...);	foo (ef ...);
f is a Pack-like type	foo (e .f . [:]. ...);	foo (e . [:].f .[:]. ...);	foo (e .f . [:]. ...);
f is not expanded	foo (e .f ...);	foo (e . [:].f ...);	foo (e .f);

The only two valid cells in that table in C++20 are the bottom-left and bottom-right ones. Note that every cell has different syntax, by design.

§ 2.10 Nested pack expansions

In order for the above table to work at all, we also need a new kind of pack expansion. When C++11 introduced pack expansion, the rules were very simple: The expression in `expr ...` must contain at least one unexpanded pack expression and every unexpanded pack expression must have the same length.

But with the concepts introduced in this proposal, we have the ability to introduce new things that behave like unexpanded pack expressions within an unexpanded pack expression and we need to define rules for that. Consider:

```
template <typename... Ts>
void foo(Ts... e) {
    bar(e.[:].... );
}

// what does this do?
foo(xstd::tuple{1}, xstd::tuple{2, 3});
```

Following the rules presented above, e `.[:]` adds a layer of packness to each element in the pack (which is fine because `xstd::tuple`s are pack-like types which define an `operator` `...()`). But what then do the `...`s refer to?

We say that adding layer of packness in the middle of an existing unexpanded pack expression will hang a new, nested unexpanded pack expression onto that.

In the above example, `e` is an unexpanded pack expression. `e.[0]` is a nested unexpanded pack expression underneath `e`.

When we encounter the the first `...`, we say that it expands the most nested unexpanded pack expression that of the expression that it refers to. The most nested unexpanded pack expression here is `e.[0]`, which transforms the expression into:

```
bar((e.[0], e.[1], e.[2], /* etc. */ e.[M-1])...);
```

This isn't really valid C++ code (or, worse, it actually is valid but would use the comma operator rather than having `M` arguments). But the idea is we now have one more `...` which now has a single unexpanded pack expression to be expanded, which is the unexpanded pack expression that expands each element in a pack-like type.

A different way of looking at is the outer-most `...` expands the outer-most unexpanded pack expression, keeping the inner ones in tact. If we only touch the outer-most `...`, we end up with the following transformation:

```
bar(e_0.[:]..., e_1.[:]..., /* etc. */ e_{N-1}.[:]...);
```

The two interpretations are isomorphic, though the latter is likely easier to understand.

Either way, the full answer to what does `foo(xstd::tuple{1}, xstd::tuple{2, 3})` do in this example is that it calls `bar(1, 2, 3)`.

For more concrete examples, here is a generalized `apply` which can take many tuples and expand them in order:

```
template <typename F, typename... Tuples>
constexpr decltype(auto) apply(F&& f, Tuples&&... tuples) {
    return std::invoke(
        std::forward<F>(f),
        std::forward<Tuples>(tuples).[:]... );
}
```

which, again more concretely, expands into something like:

```
template <typename F, typename T_0, typename T_1, ..., typename T_{N-1}>
constexpr decltype(auto) apply(F&& f, T_0 t_0, T_1 t_1, ..., T_{N-1} t_{N-1}) {
    return std::invoke(std::forward<F>(f),
        std::forward<T_0>(t_0).[:]...,
        std::forward<T_1>(t_1).[:]...,
        ...,
        std::forward<T_{N-1}>(t_{N-1}).[:]...);
}
```

And then we unpack each of these `...`s through the appropriate `operator...`s.

Similarly, `tuple_cat` would be:

```
template <typename... Tuples>
constexpr std::tuple<Tuples.[:]...> tuple_cat(Tuples&&... tuples) {
    return {std::forward<Tuples>(tuples).[:]... };
}
```

And itself leads to a different implementation of generalized `apply`:

```
template <typename F, typename... Tuples>
constexpr decltype(auto) apply(F&& f, Tuples&&... tuples) {
    return std::invoke(
        std::forward<F>(f),
        tuple_cat(std::forward<Tuples>(tuples)...).[:]...
    );
}
```

Admittedly, six `...`s is a little cryptic. But is it any worse than the current implementation?

§ 2.11 Structured bindings for pack-expandable types

Structured bindings [P0144R2] were a great usability feature introduced in C++17, but it's quite cumbersome to opt-in to the customization mechanism: you need to specialize `std::tuple_size`, `std::tuple_element`, and provide a `get()` of some sort. There was a proposal to reduce the customization mechanism by dropping `std::tuple_element` [P1096R0], which was... close. 13-7 in San Diego.

But the mechanisms presented in this paper provide a better customization point for structured bindings: `operator ...!`

This is a single function that the language can examine to determine the arity, the types, and the values. All without even having to include

```
<tuple>:
```

```
namespace xstd {
    template <typename... Ts>
    class tuple {
    public:
#ifdef ADD_ALIAS
        using ... = Ts;
#endif
        Ts& operator ...() & { return elems; }
    private:
        Ts... elems;
    };
}

int n = 0;

xstd::tuple<int, int&> tref{n, n};
auto& [i, iref] = tref;
```

This paper proposes that the above is well-formed, with or without `ADD_ALIAS` defined. And either way,

`decltype (i)` is `int`. If there is no pack alias declared, then the type will be determined from the `operator ...` result (similar to what [P1096R0] proposed). If there is a pack alias declared, then the type will be determined from that pack alias. That is, `decltype (iref)` is `int` & if `ADD_ALIAS` is defined and `int` otherwise.

More specifically, the type of the `i`th binding is `E` `[I]` if that is a valid expression (i.e. if `using ...` is declared), otherwise `std::remove_reference_t < decltype (e [I])>`.

§ 2.12 Language arrays and types with all-public members

Structured bindings works by default with language arrays and types with all- public members. This paper proposes that such types also have an implicitly-defaulted pack alias and pack operator. This allows for the same kind seamless unpacking this paper demonstrates for

```
xstd::tuple:
```

```
void bar(int, int);

int values[] = {42, 17};
bar(values[:]); // equivalent to bar(42, 17)
```

And likewise for those other types that we can already use with structured bindings:

```
struct X { int i, j; };
struct Y { int k, m; };

int sum(X x, Y y) {
    // equivalent to: return x.i + x.j + y.k * y.k + y.m * y.m
    return (x[:] + ...) + ((y[:] * y[:]) + ...);
}
```

But also provides a direct solution to the fixed-size pack problem [N4072]:

```
template <typename T, int N>
class Vector {
public:
    // I want this to be constructible from exactly N T's. The type T[N]
    // expands directly into that
    Vector(T[N][:]... vals);

    // ... which possibly reads better if you take an alias first
    using D = T[N];
    Vector(D[:]:... vals);
};
```

Note that this behaves differently from the homogenous variadic function packs paper [P1219R1]:

```
template <typename T, int N>
class Vector2 {
public:
    // independently deduces each ts and requires that
    // they all deduce to T. As opposed to the previous
    // implementation which behaves as if the constructor
    // were not a template, just that it took N T's.
    Vector2(T... ts) requires (sizeof...(ts) == N);
};
```

For instance:

```
Vector<int, 2> x('a', 2); // ok
Vector2<int, 2> y('a', 2); // ill-formed, deduction failure
```

§ 2.13 Declaring packs in other contexts

[P0780R2] allowed pack expansion in lambda init-capture:

```
template <typename... T>
void f(T... t)
{
    [...u=t]{};
}
```

If you think of lambda init-capture as basically a variable declaration with implicit `auto`, then expanding out that capture gets us to a variable declaration pack:

```
auto ...u = t;
```

This paper proposes actually allowing that declaration form.

That is, a variable pack declaration takes as its initializer an unexpanded pack. Which could be a normal pack, or it could be a pack-like type with packness added to it:

```
xstd::tuple x{1, 2, 3};

// ill-formed, x is not an unexpanded pack
auto ...bad = x;

// proposed ok
auto ...good = x[:]; // a pack of {1, 2, 3}

// proposed ok
auto ...doubles = x[:] * 2; // a pack of {2, 4, 6}
```

The same idea can be used for declaring a pack alias (as was already shown earlier in this paper):

```
template <typename... T>
struct X {
    // proposed ok
    using ...pointers = T*;
};

// ill-formed, not an unexpanded pack
using ...bad = xstd::tuple<int, double>;

// proposed ok
using ...good = xstd::tuple<int, double>[:]; // a pack of {int, double}
```

§ 2.14 Declaring packs from a *braced-init-list*

It may be worth generalizing even further and allowing declarations not just from unexpanded pack expressions but also from a *braced-init-list*. [P0341R0] presented something like this idea for variables, but instead used angle brackets for types. This paper proposed a *braced-init-list* for both cases, but with a slightly different formulation:

```
int ...squares = {1, 4, 9};
using ...types = {int, char, double};
```

gcc already uses this notation in its compile errors:

```
In instantiation of 'void foo(T ...) [with T = {int, int, int}]':
```

Such declarations would lead to the ability to provide default arguments for template parameter packs:

```
template <typename... Ts = {int}>
void f();

f(); // calls f<int>
```

And provides a direction for disambiguating expansion statements [P1306R1] over a pack:

```
template <typename... Ts>
void f(Ts... ts)
{
    for ... (auto x : {ts...})
    {
    }
}
```

The latter has some added wrinkles since you can already have a range-based for statement over a *braced-init-list* if it can be deduced to some `std::initializer_list<T>`, and we'd need a different declaration for the range (`auto range = {ts...};` vs `auto range = {ts...};`), but we *already* have a different declaration for the range in the `constexpr` case, so what's a third special case, really?

This does introduce some more added subtlety with initialization:

```
auto a = {1, 2, 3}; // a is a std::initializer_list<int>
auto... b = {1, 2, 3}; // b is a pack of int's
```

Those two declarations are very different. But also, they look very different - one has `...` and the other does not. One looks like it is declaring an object and the other looks like it is declaring a pack. This doesn't seem inherently problematic.

§ 2.15 Generalized Slicing and a simplified Boost.Mp11

This paper proposes `τ` `.[:]` to be a sigil to add packness. This also allows for more fine-grained control over which part of the pack is referenced.

Similar to Python's syntax for slicing, this paper proposes to provide indexes on one side or the other of the `:` to take just parts of the pack. For instance, `τ` `.[ 1 :]` is all but the first element of the pack. `τ` `.[ :- 1]` is all but the last element of the pack. `τ` `.[ 2 : 3]` is a pack consisting only of the third element.

Such a feature would provide an easy way to write a `std::visit` that takes the variants first and the function last:

```
template <typename... Args>
constexpr decltype(auto) better_visit(Args&&... args) {
    return std::visit(
        // the function first
        std::forward<Args...>[-1](args...[-1]),
        // all the variants next
        // note that both slices on both Args and args are necessary, otherwise
        // we end up with two packs of different sizes that need to get expanded
        std::forward<Args...>[: -1](args...[: -1])...);
}
```

Recall that since `Args` is a pack already, we index into it with `Args...[I]` rather than `Args.[I]` (which would index into each pack-like type of `Args`).

It would also allow for a single-overload variadic fold:

```
template <typename F, typename Z, typename... Ts>
constexpr Z fold(F f, Z z, Ts... rest)
{
    if constexpr (sizeof...(rest) == 0) {
        return z;
    } else {
        // we need to invoke f on z and the first elem in rest...
        // and recurse, passing the rest of rest...
        return fold(f,
            f(z, rest...[0]),
            rest...[1:]...);
    }
}
```

```

        rest...[1:]...);

    // alternate formulation
    auto head = rest...[0];
    auto ...tail = rest...[1:];
    return fold(f, f(z, head), tail...);
}

```

§ 2.15.1 Min

Andrew Sutton in a CppCon 2019 talk [[Sutton](#)] showed an example using an expansion statement to find the minimum of a pack. This paper allows for a more direct implementation:

Sutton	This proposal
<pre> template <typename... Ts> auto min_arg(Ts... args) { auto min = head(args...); template for (auto x : tail(args...)) { if (x < min) { min = x; } } return min; } </pre>	<pre> template <typename... Ts> auto min_arg(Ts... args) { auto min = args...[0]; template for (auto x : {args.[1:]...}) { if (x < min) { min = x; } } return min; } </pre>

Looks basically the same. It's just that there are no added head and tail function templates to instantiate, the latter of which has to return some kind of tuple.

§ 2.15.2 Boost.Mp11

Boost.Mp11 works by treating any variadic class template as a type list and providing operations that just work. A common pattern in the implementation of many of the metafunctions is to indirect to a class template specialization to do the pattern matching on the pack. This paper provides a more direct way to implement many of the facilities.

We just need one helper that we will reuse every time (as opposed to each metafunction needing its own helper):

```

template <class L> struct pack_impl;
template <template <class...> class L, class... Ts>
struct pack_impl<L<Ts...>> {
    // a pack alias for the template arguments
    using ... = Ts;

    // an alias template for the class template itself
    template <typename... Us> using apply = L<Us...>;
};

template <class L, class... Us>
using apply_pack_impl = typename pack_impl<L>::template apply<Us...>;

```

Boost.Mp11	This proposal
<pre> template <class L> struct mp_front_impl; template <template <class...> class L, class T, class... Ts> struct mp_front_impl<L<T, Ts...>> { using type = T; }; template <class L> using mp_front = typename mp_front_impl<L>::type; </pre>	<pre> template <class L> using mp_front = pack_impl<L>.[0]; </pre>
<pre> template <class L> struct mp_pop_front_impl; template <template <class...> class L, class T, class... Ts> struct mp_pop_front_impl<L<T, Ts...>> { using type = T; }; </pre>	<pre> template <class L> using mp_pop_front = apply_pack_impl< L, pack_impl<L>.[1:]...>; </pre>

Boost.Mp11	This proposal
<pre>}; template <class L> using mp_pop_front = typename mp_pop_front_impl<L>::type;</pre>	
<pre>// you get the idea template <class L> using mp_second = /* ... */; template <class L> using mp_third = /* ... */; template <class L, class... Ts> using mp_push_front = /* ... */; template <class L, class... Ts> using mp_push_back = /* ... */; template <class L, class T> using mp_replace_front = /* ... */; // ...</pre>	<pre>template <class L> using mp_second = pack_impl<L>.[1]; template <class L> using mp_third = pack_impl<L>.[2]; template <class L, class... Ts> using mp_push_front = apply_pack_impl< L, Ts..., pack_impl<L>.[:]...>; template <class L, class... Ts> using mp_push_back = apply_pack_impl< L, pack_impl<L>.[:]..., Ts...>; template <class L, class T> using mp_replace_front = apply_pack_impl< L, T, pack_impl<L>.[1:]...>;</pre>

§ 2.16 Implementing variant

While most of this paper has dealt specifically with making a better tuple, the features proposed in this paper would also make it much easier to implement variant as well. One of the difficulties with variant implementations is that you need to have a `union`. With this proposal, we can declare a variant pack too.

Here are some parts of a variant implementation, to demonstrate what that might look like. Still need *some* metaprogramming facilities, but it's certainly a lot easier.

```
template <typename... Ts>
class variant {
    int index_;
    union {
        Ts... alts_;
    };
public:
    constexpr variant() requires std::default_constructible<Ts...[0]>
        : index_(0)
        , alts_...[0]()
    { }

    ~variant() requires (std::is_trivially_destructible<Ts> && ...) = default;
    ~variant() {
        mp_with_index<sizeof...(Ts)>(index_,
            [](auto I){ destroy_at(&alts_...[I]); });
    }
};

template <size_t I, typename T>
struct variant_alternative;

template <size_t I, typename... Ts>
    requires (I < sizeof...(Ts))
struct variant_alternative<I, variant<Ts...>> {
    using type = Ts...[I];
};

template <size_t I, typename... Types>
constexpr variant_alternative_t<I, variant<Types...>>*
get_if(variant<Types...>* v) noexcept {
```

```

        if (v.index_ == I) {
            return &v.alts_...[I];
        } else {
            return nullptr;
        }
    }
}

```

Directly indexing into the union variant members makes the implementation much easier to write and read. Not needing a recursive union template is a nice bonus.

§ 2.17 Enumerating over a pack

As another example, let's say we want to take a parameter pack and print its contents along with an index. Here are some ways we could do that with this proposal (assuming expansion statements):

```

// iterate over the indices, and index into the pack
template <typename... Ts>
void enumerate1(Ts... ts)
{
    for ... (constexpr auto I : view::iota(0u, sizeof...(Ts))) {
        cout << I << ' ' << ts...[I] << '\n';
    }
}

// construct a new pack of tuples
template <typename... Ts>
void enumerate2(Ts... ts)
{
    constexpr auto indices = view::iota(0u, sizeof...(Ts));
    auto enumerated = tuple{tuple{indices.[:], ts}...};
    for ... (auto [i, t] : enumerated) {
        cout << i << ' ' << t << '\n';
    }
}

```

§ 2.18 What about Reflection?

Two recent reflection papers ([P1240R0] and [P1717R0]) provide solutions for some of the problems this paper is attempting to solve. What follows is my best attempt to compare the reflection solutions to the generalized pack solutions presented here. I am not entirely sure about the examples on the left, but hopefully they are at least close enough to correct to be able to evaluate the differences.

Reflection	This proposal
<pre> // member pack declaration (P1717) template <typename... Types> class tuple { consteval { int counter = 0; for... (meta::info type : reflexpr(Types)) { auto fragment = __fragment struct { typename(type) unqualid("element_", counter); }; -> fragment; ++counter; } } }; </pre>	<pre> template <typename... Types> class tuple { Types... element; }; </pre>
<pre> // pack indexing (P1240) template <size_t I, typename... Ts> using at = typename(std::vector{reflexpr(Ts)...}[I]); </pre>	<pre> template <size_t I, typename... Ts> using at = Ts...[I]; </pre>
<pre> // generalized pack indexing (P1240) template <typename... Types> </pre>	<pre> template <typename... Types> struct tuple { </pre>

Reflection	This proposal
<pre>struct tuple { consteval static auto types() { return std::vector{reflexpr(Types)...}; } }; template <size_t I, typename T> using tuple_element_t = typename(T::types() [I]);</pre>	<pre>using ... = Types; }; template <size_t I, typename T> using tuple_element_t = T.[I];</pre>
<pre>template <typename... Types> struct tuple { consteval auto members() const { // return some range of vector<meta::info> // here that represents the data members. // I am not sure how to implement that } }; template <typename Tuple> void call_f(Tuple const& t) { f(t.unreflexpr(t.members())...); }</pre>	<pre>template <typename... Types> struct tuple { operator Types const&...() const& { return elems; } Types... elems; }; template <typename Tuple> void call_f(Tuple const& t) { return f(t.[...]); }</pre>

It's not that I think that the reflection direction is bad, or isn't useful. Far from. Indeed, this paper will build on it shortly. It's just that dealing with tuple is, in no small part, an ergonomics problem and I don't think any reflection proposal that I've seen so far can adequately address that. This is fine - sometimes we need a specific language feature for a specific use-case.

§ 3 The Pair Example and others

As shocking as it might be to hear, there are in fact other types in the standard library that are not `std::tuple<Ts...>`. We should probably consider how to fit those other types into this new world.

§ 3.1 `std::pair`

It might seem strange, in a paper proposing language features to make it easier to manipulate packs, to start with tuple (the quintessential pack example) and then transition to pair (a type that has no packs). But pair is in many ways just another tuple, so it should be usable in the same ways. If we will be able to inline unpack a tuple and call a function with its arguments, it would be somewhat jarring if we couldn't do the same thing with a pair. So how do we?

In the previous section, this paper laid out proposals to declare a variable pack, to provide for an alias pack and `operator...`, to index into each, and connect all of this into structured bindings. How would this work if we do *not* have a pack anywhere?

```
namespace xstd {
    template <typename T, typename U>
    struct pair {
        T first;
        U second;

        using ... = ???;
        operator ??? ... () { return ???; }
    };
}
```

The direction this paper proposes is a recursive one. I'm taking two ideas that are already present in the language and merging them:

- `operator->()` recurses down until it finds a pointer
- Expansion statements [\[P1306R1\]](#) can take either an unexpanded pack, a type that adheres to the structured bindings protocol, or a `constexpr` range.

In the same vein, this paper proposes that both the pack operator and pack aliases can be defined in terms of an unexpanded pack or a type that defines one of these aliases:

```

namespace xstd {
    template <typename T, typename U>
    struct pair {
        T first;
        U second;

        using ... = tuple<T, U>;
        tuple<T&, U&> operator ... () & { return {first, second}; }
    };
}

```

With the definition of `xstd::tuple` presented in this paper, this is now a light type to instantiate (or at least, as light as possible), so the extra overhead might not be a concern.

In the following example:

```

void f(int&, char&);
xstd::pair<int, char> p{1, 'x'};
f(p.[:]);

```

`p.[:]` will invoke `p.operator ...()`, which gives a tuple `<int&, char&>`. That is not a pack, so we invoke its `operator ...()`, which gives us a pack of `int&` and `char&`.

A different, non-recursive approach would be to use the reflection facilities introduced in [\[P1240R0\]](#) and allow the returning of a consteval range:

```

namespace xstd {
    template <typename T, typename U>
    struct pair {
        T first;
        U second;

        using ... = typename(std::vector{reflexpr(T), reflexpr(U)});

        consteval auto operator...() const {
            return std::vector{
                reflexpr(first),
                reflexpr(second)
            };
        }
    };
}

```

In the above example, `f(p.[:])` would evaluate as `f(unreflexpr(p.operator...()))`.

It's not clear if this direction will actually work, since you would have to disambiguate between the case where you want the identifiers and the case where actually you want the meta-`info` objects themselves. Let's call it an open question.

Of course, for this particular example, both the pack alias and operator could be defaulted.

§ 3.2 `std::integer_sequence` and Ranges

There is a paper in the pre-Cologne mailing specifically wanting to opt `std::integer_sequence` into expansion statements [\[P1789R0\]](#). We could instead be able to opt it into the new pack protocol:

```

template <class T, T... Ints>
struct integer_sequence {
    std::integral_constant<T, Ints> operator ...() const {
        return {};
    }
};

```

One of the things we could have built on top of expansion statements was to implement tuple swap like so [\[Stone.Swap\]](#):

```

template <class... TYPES>
constexpr
void tuple<TYPES...>::swap(tuple& other)
    noexcept((is_nothrow_swappable_v<TYPES> and ...))

```

```

{
    for...(constexpr size_t N : view::iota(0u, sizeof...(TYPES))) {
        swap(get<N>(*this), get<N>(other));
    }
}

```

But what if I wanted to unpack a range into a function? It doesn't seem so far fetched that if you can use an expansion statement over a range (which requires a known fixed size) that you should be able to use other language constructs that also require a known fixed size: structured bindings and tuple unpacking:

```

auto [a, b, c] = view::iota(0, 3); // maybe this should work
foo(view::iota(0, 3).[:]....);    // ... and this too

```

§ 4 Proposal

All the separate bits and pieces of this proposal have been presented one step at a time during the course of this paper. This section will formalize all the important notions.

§ 4.1 Pack declarations

You can declare member variable packs, namespace-scope variable packs, and block-scope variable packs. You can declare alias packs. The initializer for a pack declaration has to be either an unexpanded pack or a *braced-init-list*.

These can be directly unpacked when in non-dependent contexts.

You can declare packs within structured binding declarations (this paper may we well just subsume [\[P1061R0\]](#)).

§ 4.2 Pack-like type

To start with, structured bindings today works on three kinds of types:

1. Arrays
2. Tuple-Like types – defined as types that satisfy `std::tuple_size<E>`, `std::tuple_element<i, E>`, and `get<i>()`. Note that this is a language feature that nevertheless has a library hook.
3. Types that have all of their non-static data members in the same class. This one is a little fuzzy because it's based on accessibility.

This paper proposes the notion of a *pack-like type*. A pack-like type:

1. Is an array type
2. Has one of:
 - a. an unnamed pack alias type that names a pack-like type or a pack
 - b. a pack operator that returns a pack-like type or a pack

If any of these new special named members yields a reflection range, that range will be reified as appropriate before further consideration. If a type provides only a pack alias, it can be indexed into/unpacked as a type but not as a value. If a type provides only a pack operator, it can be indexed/unpacked as a value but not as a type.

3. Is a Tuple-Like type (as per structured bindings)
4. Is a constexpr range (the constexpr-ness is important because of the fixed size)
5. Is a type that has all of its non-static data members in the same class (as per structured bindings)

This paper proposes to redefine both structured bindings and expansion statements in terms of the pack-like type concept, unifying the two ideas. Any pack-like type can be expanded over or used as the right-hand side of a structured binding declaration.

Any pack-like type can be indexed into, `τ[i]` will yield the *i*th type (if *τ* is a type) or *i*th value (if *τ* is a variable) of the type. Any pack like type can be sliced and unpacked via `τ[:]....` or with specific indices.

This also unifies the special unpacking rules in [\[P1240R0\]](#): a reflection range is a pack-like type, therefore it can be unpacked.

§ 4.3 Dependent packs

Member packs and block scope packs can be directly unpacked when in non-dependent contexts. We know what they are.

In dependent contexts, anything that is not a pack must be explicitly identified as a pack in order to be treated as one. Similar to how we need the `typename` and `template` keyword in many places to identify that such and such an expression is a type or a template, a preceding `...` (or whatever alternate spelling) will identify the expression that follows it as a pack. If that entity is *not* a pack, then the indexing or unpacking expression is ill-formed.

§ 5 Acknowledgments

This paper would not exist without many thorough conversations with Agustín Bergé, Matt Calabrese, and Richard Smith. Thank you.

Thank you to David Stone for pointing out many issues.

§ 6 References

[Boost.Mp11] Peter Dimov. 2017. Boost.Mp11: A C++11 metaprogramming library - 1.70.0.

https://www.boost.org/doc/libs/1_70_0/libs/mp11/doc/html/mp11.html

[Calabrese.Argot] Matt Calabrese. 2018. C++Now 2018: Argot: Simplifying Variants, Tuples, and Futures.

https://www.youtube.com/watch?v=pKVCB_Bzalk

[N3728] Mike Spertus. 2013. Packaging Parameter Packs (Rev. 2).

<https://wg21.link/n3728>

[N4072] Maurice Bos. 2014. Fixed Size Parameter Packs.

<https://wg21.link/n4072>

[N4191] A. Sutton, R. Smith. 2014. Folding expressions.

<https://wg21.link/n4191>

[N4235] Daveed Vandevoorde. 2014. Selecting from Parameter Packs.

<https://wg21.link/n4235>

[P0144R2] Herb Sutter. 2016. Structured Bindings.

<https://wg21.link/p0144r2>

[P0195R2] Robert Haberlach, Richard Smith. 2016. Pack expansions in *using-declarations*.

<https://wg21.link/p0195r2>

[P0341R0] Mike Spertus. 2016. parameter packs outside of templates.

<https://wg21.link/p0341r0>

[P0535R0] Matthew Woehlke. 2017. Generalized Unpacking and Parameter Pack Slicing.

<https://wg21.link/p0535r0>

[P0780R2] Barry Revzin. 2018. Allow pack expansion in lambda init-capture.

<https://wg21.link/p0780r2>

[P0847R2] Gašper Ažman, Simon Brand, Ben Deane, Barry Revzin. 2019. Deducing this.

<https://wg21.link/p0847r2>

[P1061R0] Barry Revzin, Jonathan Wakely. 2018. Structured Bindings can introduce a Pack.

<https://wg21.link/p1061r0>

[P1096R0] Timur Doumler. 2018. Simplify the customization point for structured bindings.

<https://wg21.link/p1096r0>

[P1219R1] James Touton. 2019. Homogeneous variadic function parameters.

<https://wg21.link/p1219r1>

- [P1240R0] Andrew Sutton, Faisal Vali, Daveed Vandevoorde. 2018. Scalable Reflection in C++.
<https://wg21.link/p1240r0>
- [P1306R1] Andrew Sutton, Sam Goodrick, Daveed Vandevoorde. 2019. Expansion statements.
<https://wg21.link/p1306r1>
- [P1717R0] Andrew Sutton, Wyatt Childers. 2019. Compile-time Metaprogramming in C++.
<https://wg21.link/p1717r0>
- [P1789R0] Alisdair Meredith. 2019. Library Support for Expansion Statements.
<https://wg21.link/p1789r0>
- [Smith.Pack] Richard Smith. 2013. A problem with generalized lambda captures and pack expansion.
<https://groups.google.com/a/isocpp.org/d/msg/std-discussion/ePRzn4K7VcM/Cvy8M8EL3YAJ>
- [Stone.Swap] David Stone. 2019. Library Support for Expansion Statements: P1789.
<http://lists.isocpp.org/lib-ext/2019/06/11932.php>
- [Sutton] Andrew Sutton. 2019. Meta++: Language Support for Advanced Generative Programming.
<https://youtu.be/kjQXhuPX-Ac?t=389>